Page No.:
Date:

3. Synchronization

Q.1 What is physical clocks? Explain any one physical clock synchronization method.

⇒
- Clock synchronization refers to the process of ensuring that all clocks across various nodes or computers in the system are set to that same time or at least have their time closely aligned.
- In distributed systems each node operates with its clock, which can lead to differences.
- However, the goal of physical clock synchronization is to overcome this challenge.
- This involves equipping each node with a clock that is adjusted to match Universe Coordinated Time (UTC) a recognized standard.
- Key reasons synchronization is crucial:

1) Data consistency

⇒ Synchronization maintains data accuracy across multiple concurrent processes.
- It prevents data corruption by ensuring that only one process modifies data at a time.

2) Deadlock prevention

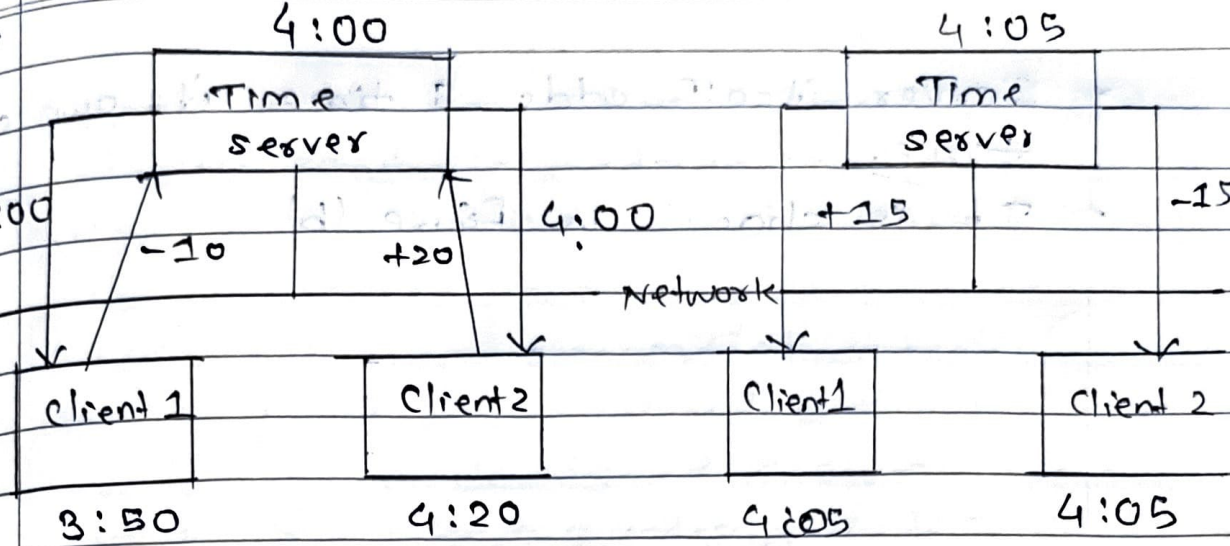
⇒ Proper synchronization help prevent deadlock.
- Deadlock occurs when two or more processes are each waiting for the other to release resources.

3) Efficient resource sharing
⇒ Synchronization allows multiple threads or processes to share resources without interference.
- It coordinates access to shared resources, such as database or files, to optimize performance.

4) Orderly Execution
⇒ Synchronization ensures that critical sections of code run in a specified order.
- This order is necessary when the output of one process depends on the input of another.

- Physical Clock Synchronization Methods :-
  - 1) Cristian's algorithm
  - 2) Berkeley algorithm.
  - 3) NTP (Network Time Protocol)

- Berkeley Algorithm
- In Berkeley UNIX, time daemon is active as it polls every machine asking current time there.
- It then calculates average time of received answers from all machines.
- It then tells other machines either advance their clocks to new time or slow their clocks until specified time is set.



- As shown in above figure (a), initially time daemon has time value of 4:00.
- Client 1 has clock time value 3:50 and client 2 has clock time value of 4:20.
- Time server asks to other machines about their current time including it.
- Client 1 replies that, it is lagging by 10 w.r.t server time which is 4:00.
- Similarly, client 2 replies that, it is leading by 20 w.r.t server time which is 4:00.
- Server will reply as 0 to itself.
- Meanwhile server time increases to 4:05.
- Therefore server sends client 1 and 2 messages about how to adjust their clocks.
- It tells client 1 to advance clock time by 15.
- Similarly, it tells client 2 to decrement the time value by 15.

- Server itself adds 5 time its own clock time.
- It is shown in figure (b)

- Advantages:

1) Centralized time coordinator
⇒ Berkeley algorithm uses a centralized time coordinator, which responsible for maintaining global time and distributing it to the all client machines.

2) Clock Adjustment
⇒ The algorithm adjusts the clock of each client machine based on the differences between its local time and the time received from the time coordinator.

3) Fault tolerance
⇒ Berkeley algorithm is fault-tolerant, as it can handle failures in the network or the time coordinator by using backup time coordinators.

Page No.:
Date:

Q.2 What is logical clocks? Why are logical clocks required in distributed systems? How Lamport does synchronization logical clock? Which events are said to be concurrent in Lamport timestamp's.

⇒

- A logical clock is an abstract mechanism used to order events in a distributed system where there is no global physical clocks.
- Many algorithms works based on the internal consistency of clocks and not with real-time.
- For these algorithms, clocks are said to be logical clocks.
- Logical clocks are required to:
  i) Maintain event ordering.
  ii) Detect concurrent events
  iii) Ensure consistency in event logs.
  iv) Coordinate actions like mutual exclusion, deadlock detection, etc.

- Lamport's logical clock

- The algorithm of Lamport timestamp is a simple algorithm used to determine the order of events in a distributed system.
- As different nodes or processes will typically not be perfectly synchronized, this algorithm is used to provide partial ordering of events with minimal overhead, and provide a starting point for the more advanced vector clock method.

Page No.:
Date:

- For synchronization of logical clocks, Lamport established a relation term as "happen-before" relation.

1) Happen-before relation is a transitive relation, therefore if $p \to q$ and $q \to r$, then $p \to r$
2) If two events a and b occur in different processes which not at all exchange message amongst them, then $a \to b$ is not true, but neither is $b \to a$ which is antisymmetry.

- The algorithm follows some simple rules:

i) A process increments its counter before each event in that process.
ii) When a process sends message, it include its counter value with the message.
iii) On receiving message, the counter of receipient is updated, if necessary, to the greater of its current counter and the timestamp in the received message. The counter is then incremented by 1 before the message is considered received.

- Example how lamport synchronizes clocks:

P_1		P_2		P_3
0		0		0
6	m_1	8		10
12		16		20
18		24	m_2	30
24		32		40
30		40		50
36		48		60
42		56	m_3	70
48		64		80
54	m_4	72		90
60		80		100

a) Three processes, each with its own clock.

P_1		P_2		P_3
0		0		0
6	m_1	8		10
12		16		20
18		24	m_2	30
24		32		40
30		40		50
36		48		60
42	(61)	56	m_3	70
48		69		80
~~70~~	m_4	~~77~~		90
76		85		100

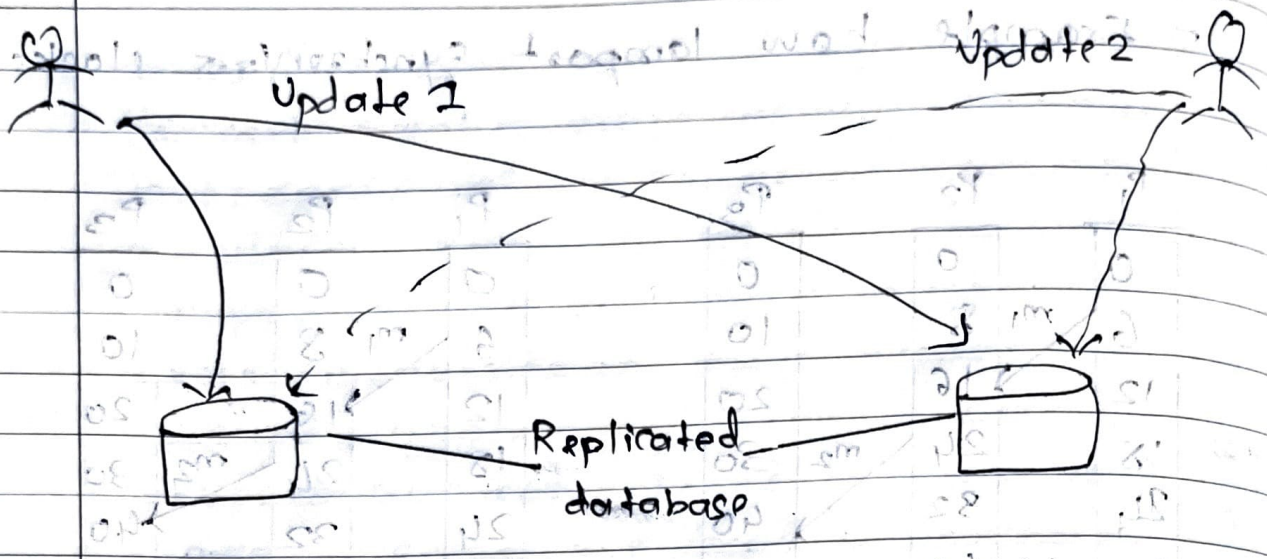
b) Lamport's algorithm corrects the clock

- Concurrent events in lamport timestamps:

1) Totally Ordered Multicast

⇒

- We need to guarantee that concurrent updates on replicated databases are seen in the same order everywhere. This requires total-ordered multicast.
- Update 1: add 100 to an account
- Update 2: add 1% interest to account.



Update 1 is performed before update 2

Update 2 is performed before update 1

- Lamport's logical clocks can be used to implement totally-ordered multicast in a completely distributed fashion.

Page No.:
Date:

Q.3 Explain the Bully algorithm with appropriate example. State its advantages and disadvantages.

⇒

- The Bully algorithm is a popular method for electing a coordinator in distributed systems.
- When nodes or processes in a system need a leader to manage tasks or make decisions, this algorithm help them choose one, even if some nodes fails.
- The process involves nodes "bullying" each other by checking who has the highest ID, and the one with the highest ID becomes the coordinator.
- It operates as follows :

1) Initiation :

⇒ When a node detects that the current leader has failed, it initiates an election.

2) Election process

⇒ The initiating node sends an "election" message to all other nodes with higher priority.

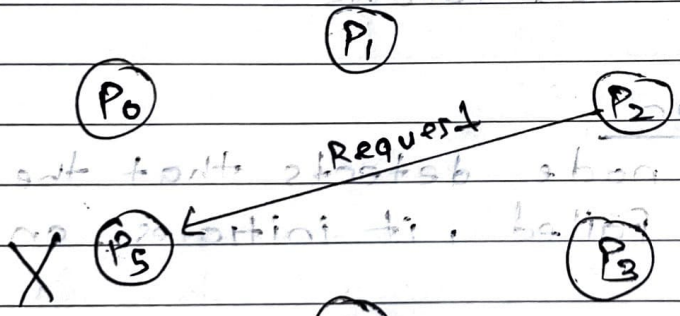
- Nodes with higher priority respond by either acknowledging their current leader status or declaring themselves candidates.
- If no higher-priority node responds, the initiating node assumes leadership.

3) Notification

⇒ The newly elected leader informs all nodes of its leadership status, ensuring consistency across the distributed system.

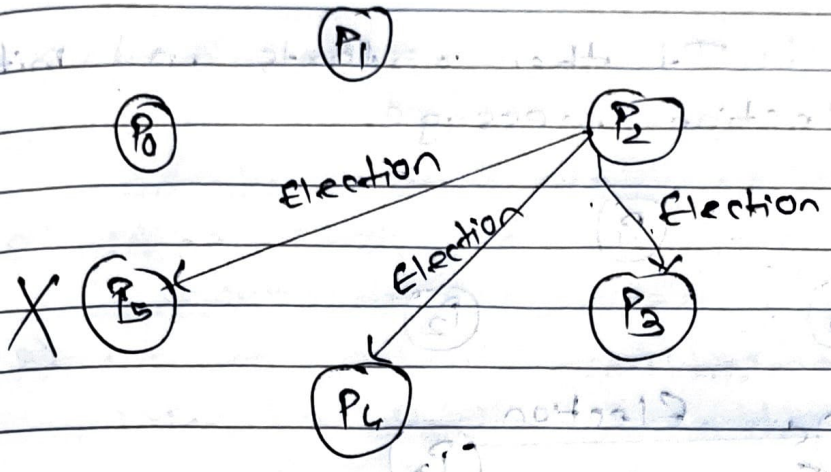
- Let's understand the steps involved in bully algorithm for leader Node election through an example.
- Let's say there are 6 processes P_0, P_1, P_2, P_3, P_4, P_5 written in ascending order of their process ID.

Step 1: Suppose process P_2 sends a message to coordinator P_5 and P_5 doesn't respond in a desired time T

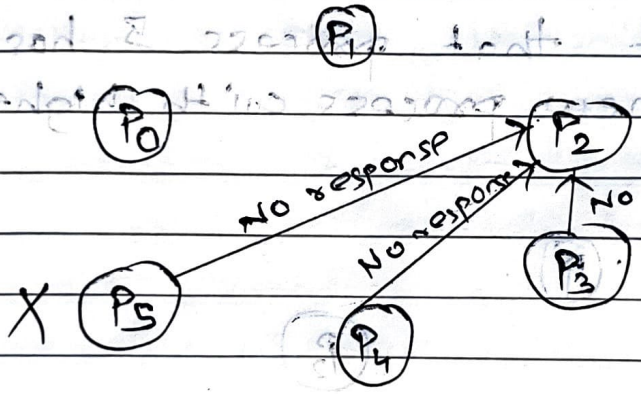


P_5 is current coordinator and currently it is crashed.

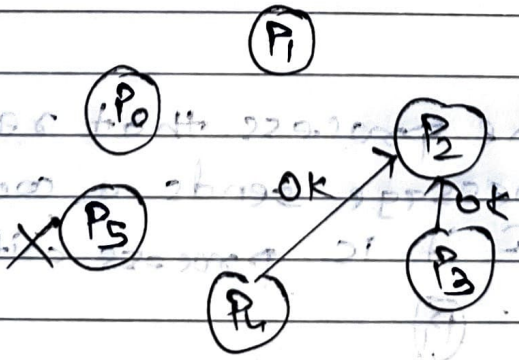
Step 2: Then process P_2 sends an election message to all processes with Process ID greater than P_2 and awaits a response from the processes.



Step 3: If no one ~~become~~ responds, P_2 wins the election and becomes the coordinator.

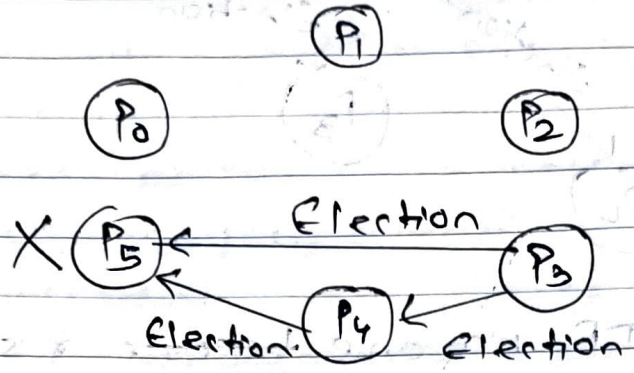


Step 4 : If any of process with ProcessID higher than 2 responds OK, P_2's job is done and this process will take over.

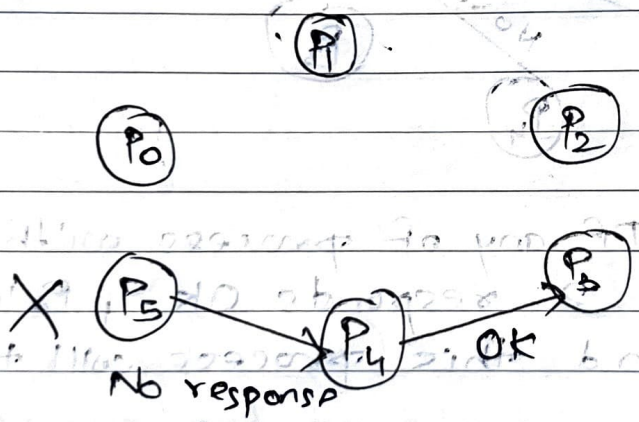


Page No.:
Date:

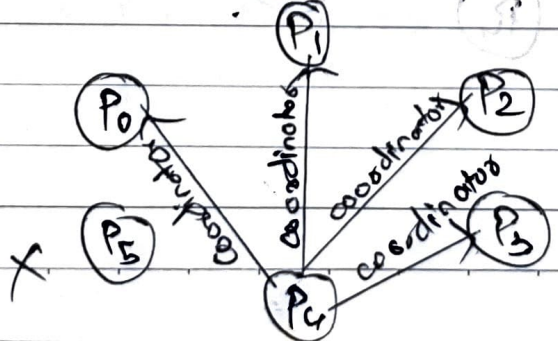
Step 5 : It then restarts and initiates an election message.



Step 6 : Process P4 responds to P3 OK message to confirm its alive and P4 figure out that process 5 has crashed, and the new process with highest ID is process 4.



Step 7 : The process that receives an election message sends a coordinator message if it is process with highest ID



Page No.:
Date:

- Advantages:

1) Simple to implement
2) Always elects with highest priority process.
3) Works even if multiple processes initiate election simultaneously.

- Disadvantage:

1) Requires many message.
2) Assumes reliable communication.
3) Does not handle network partitioning well.

Page No.:
Date:

Q.4 Explain the need of election algorithm

⇒

- An election algorithm is used in distributed system to select a leader among a group of processes.
- The leader is responsible for managing tasks, synchronization and resource allocation.
- Key requirements :

a) Uniqueness : Only one leader is selected at a time.

b) Fault tolerance : If the leader fails, a new leader should be elected.

c) Fairness : Every process should have an equal chances of becoming a leader.

d) Minimal message overhead : The algorithm should work efficiently with minimal communication.

e) Consistency : Every process should agree on the elected leader.

- Example of Election algorithm :

i) Bully – elects the process with highest ID.

ii) Ring – uses a token-passing approach in a logical ring.

Page No.:
Date:

Q.5 Differentiate between token-based algorithm and Non-token based algorithm.

=>

Token-based algorithm	Non-token based algorithm
1) In the token-based algorithm, a unique token is shared among all the sites in distributed system.	1) In non-token based algorithm, there is no token even not any concept of sharing token for access.
2) Here, a site is allowed to enter the critical section if it possesses the token.	2) Here, two or more successive rounds of message are exchanged between sites to determine which site is to enter CS next.
3) The token based algorithm uses the sequences to order the request for critical section.	3) Non-token based algorithm uses the timestamp to order the request for CS.
4) It produces less message traffic compared to non-token based algorithm.	4) It produces more message traffic as compared to token based algorithm.
5) They are free from deadlock because of existence of unique token.	5) They are not free from deadlock problem as they are based on timestamp only.
6) E.g., Raymond-based tree-based algorithm, Suzuki-Kasami algorithm	6) E.g., Lamport's algorithm, Ricart-Agrawala algorithm, Maekwa's algorithm.

Page No.:
Date:

Q.6 What is distributed mutual exclusion? Explain how Suzuki-Kasami's broadcast algorithm achieves distributed mutual exclusion.

⇒

- Distributed mutual exclusion is a mechanism to ensure that only one process among multiple distributed processes can enter the critical section at a time, without using shared memory.
- It solves the mutual exclusion problem in distributed system where:
  i) There is no global clock.
  ii) Processes communicate via message passing.
  iii) Consistency and fairness must be maintained.
- Mutual exclusion algorithms should have following features :-

1) Algorithm should not involve in deadlocks
⇒ More than two nodes in distributed system should not wait endlessly for the messages which will never be received.

2) Fairness
⇒ As there is absence of physical global clock, either requests at any node should be executed in the order of arrival or in the order they were issued.

3) Fault tolerance

⇒ In case of any failure in the course of carrying the work, if any failure occurs, the mutual exclusion algorithms should recognize itself to carry out the further designated job.

- Suzuki-Kasami's Broadcast Algorithm

- Suzuki - Kasami algorithm is a token-based algorithm for achieving mutual exclusion in distributed systems.
- This is modification of Ricart-Agrawala algorithm which uses REQUEST and REPLY messages to ensure mutual exclusion.
- Data structure and Notation :

1) An array of integers RN[1....N], A site Si keeps RNi[1...N] is the largest sequence number received so for through REQUEST message from Si.

2) An array of integers LN[1....N], This array is used by the token. LN[J] is the sequence no. of request that is recently executed by Sj.

3) A Queue Q,
This data structure is used by the token to keep record of ID of sites waiting for the token

Page No.:
Date:

- Algorithm

1) Request for CS

⇒

i) If process P_i wants to enter in CS and does not have token, then it increment its sequence number $RN_i[i]$. It then sends REQUEST(P_i, sn) message to all the processes.
In this message sn is updated value of $RN_i[i]$.

ii) After receiving this request message, process P_j sets $RN_j[i]$ to $\max(RN_j[i], sn)$. If P_j has token then it sends to P_i if

$$RN_j[i] = SN[i] + 1$$

2) Executing the CS

⇒ After receiving the token, process P_i executes in CS.

3) Existing the CS

⇒ Process P_i carry out following updates after it finishes execution of CS.

i) It sets $SN[i] = RN_i[i]$

ii) For all processes P_j, if their IDs does not belong to the token queue then these IDs in it if $RN_i[j] = SN[j] + 1$

iii) If token queue is still has some messages in it then. Delete ID at head of queue and sends token to the process of that ID.

- For e.g.,

i) Processes P1, P2, P3
ii) Initially, P1 has the token
iii) P2 and P3 request CS
iv) Requests are broadcasted
v) P1, after CS, passes token to next in line (P2), then P3.

Q.7 Discuss Raymond's tree based algorithm of token-based in distributed mutual exclusion.

⇒

- Raymond's tree-based algorithm is lock based algorithm for mutual exclusion in a distributed system.
- In this, a site is allowed to enter the critical section if it has the token.
- In this algorithm, all sites are arranged as a directed tree such that all edges of the tree are assigned direction towards the site that holds the token.
- Site which holds the token is also called root of the tree.
- It minimizes message passing by using point-to-point communication rather than broadcast.
- Algorithm Components : Each process maintains -

1) Holder - ID of the neighbor from which to token will come.
2) Request queue - queue of requests from neighboring nodes.
3) Token - Boolean flag indicating if the process has the token.

Page No.:
Date:

- **Working**

1) **Requesting the token**

⇒ IF a process P wants to enter the Critical Section (CS):

i) IF it doesnot have the token and its request queue is empty, it

a) Sends a REQUEST message to its holder.

b) Adds its own ID to its request queue.

2) **Passing the token**

⇒ The token holder, when not in CS, checks its request queue:

i) It passes the token to the first requestor in the queue.

ii) Updates its holder to the new process.

iii) Removes that process from queue.

3) **Entering and Existing CS**

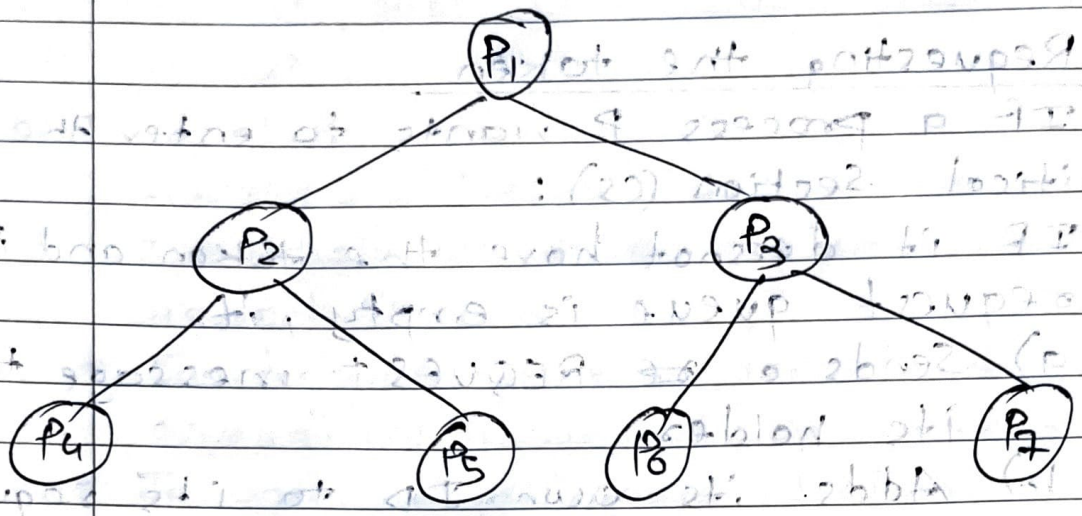
⇒ Once a process gets the token, it can enter the CS.

- After existing the CS:

i) It checks its request queue.

ii) IF the queue is non-empty, it passes the token to the next requester.

- For example,



1) P6 wants to enter CS:
⇒ P6 does not have the token.
- P6 sends a request to P3 (its parent)
- P6 adds its own ID to its request queue.

2) P3 receives the request of P6
⇒ P3 adds P6 to its request queue.
- P3 forwards request to P1

3) P1 holds the token and is not in CS:
⇒ P1 checks its queue, so it sends the TOKEN to P3.

4) P3 receives the token
⇒ P3 sees that P6 is at the front of its queue.
- P3 passes the TOKEN to P6
- P3 updates its holder to P6 and removes P6 from the queue.

5). P6 receives the token and enters the CS.

7. After P6 exits CS:

i). If P6 has pending request in its queue, it will pass the token accordingly.

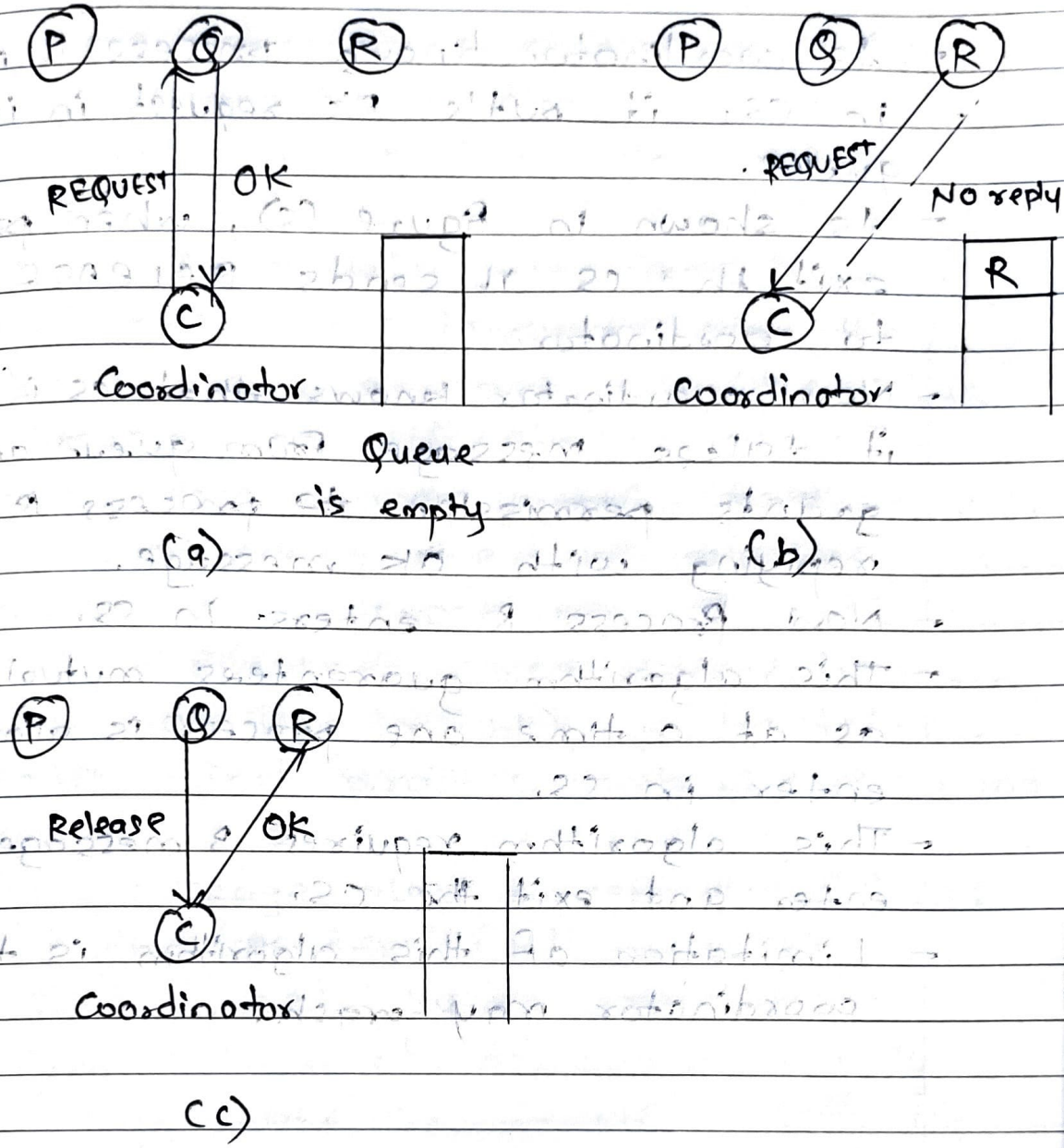
ii) Otherwise, it holds the token until another request is made.

Page No.:
Date:

Q. 8 Explain the centralized algorithms for Mutual exclusion in Distributed system.

⇒

- In this algorithm, a central coordinator process is responsible to give permission to other processes to enter in critical section (CS).
- Coordinator process keeps track of which CS is busy or currently in execution.
- Process which wants to enter in CS sends REQUEST message to coordinator stating the name of CS.
- If CS is free then coordinator sends OK message to requesting process to grant permission to enter in stated CS in message.
- If in stated CS, already other process is executing then it puts the request message in its queue.
- When currently executing process in CS exists, it sends RELEASE message to coordinator process.
- As a result coordinator knows that CS is now free and it takes request from the queue to grant permission to enter in CS.
- If requesting process is ~~blocked~~ still blocked, it unblocks and enters in CS.



- As shown in fig (a), Process C is coordinator and currently no process is executing in CS.
- Q sends REQUEST message to coordinator and it permits process Q to enter in CS by replying with OK message.
- As shown in fig (b), process R also sends REQUEST message to coordinator to get permission to enter in same CS.

- As coordinator knows process R is already in CS, it puts R's request in its queue.
- As shown in figure (c), when process Q exit the CS it sends RELEASE message to coordinator.
- Now coordinator knows that CS is free, it takes message from queue and grants permission to process R by replying with OK message.
- Now Process R enters in CS.
- This algorithm guarantees mutual exclusion as at a time one process is allowed to enter in CS.
- This algorithm requires 3 messages to enter and exit the CS.
- Limitation of this algorithm is the coordinator may crash.

Page No.:
Date:

Q.9 Explain : Ricart-Agrawala's algorithm and how it optimizes the message overhead in achieving mutual exclusion.

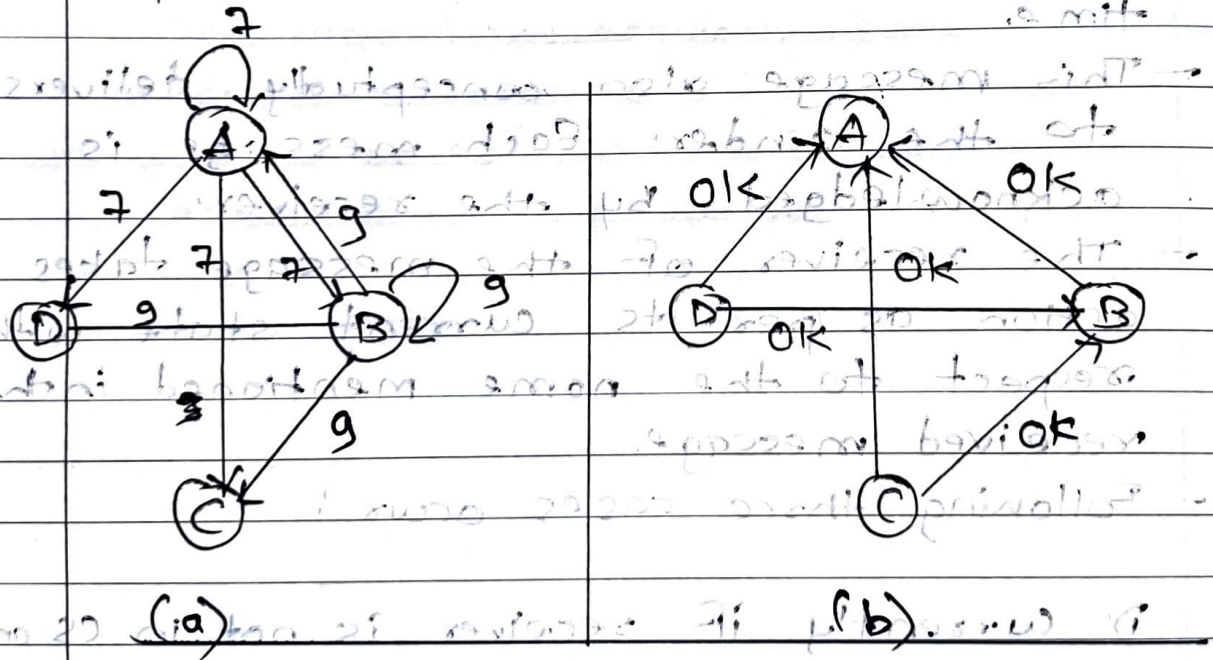
⇒

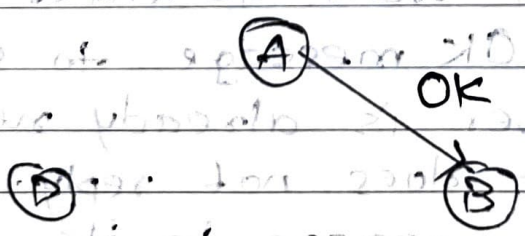
- Ricart-Agrawala's algorithm is an optimization of Lamport's algorithm.
- The working of the algorithm is as follows :
- When the process wants to enter in CS, it sends the REQUEST message to all other processes.
- This REQUEST message contains the name of CS in which it wants to enter, its process number and current time.
- This message also conceptually delivers to the sender. Each message is acknowledged by the receiver.
- The receiver of the message takes action as per its current state with respect to the name mentioned in the received message.
- Following three cases occur :

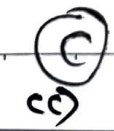
1) Currently if receiver is not in CS and also does not want to enter it then it replies OK message to sender.
2) If receiver is already running in CS then it does not reply. ~~If puts~~ It puts message in its queue.

3) If receiver is about the enter in CS and yet has not done then it compares the timestamp in incoming request message with the timestamp in request message which it has already sent to other processes. The lowest one wins.

- Receiver sends OK message to the sender if incoming request message has lower timestamp value.
- If its own request message has lower timestamp value then it puts the incoming request message in its queue and does not send any reply.







- As shown in fig (a), process A and B both wants to enter in CS simultaneously.
- Process A and B both sends REQUEST message to everyone with timestamp 7 and 9 respectively.
- As shown in fig (b), as process C and D both are not interested in entering the CS, both sends OK message as reply to sender processes A and B.
- Process A and B both see conflict and compare timestamps.
- In this case process A wins as its message timestamp 7 is lower than the timestamp of message sent by process B, which is 9.
- A process A enters in CS and keeps request of process B in its queue.
- When process A exits the CS, it sends OK message to process B so that later process B will enter in CS.
- It is shown in fig (c).
- Hence, algorithm ensures the mutual exclusion in distributed system as lower timestamp process is allowed to enter in CS in conflict situation.

How it optimizes Message Overhead ?

1) No separate RELEASE message
⇒ Ricart-Agrawala eliminates the need for a RELEASE message.
- Process reply only when needed, and deferred replies act as implicit release.

2) Peer-to-peer communication
⇒ Direct communication between requesting process and all others.
- No coordinator or central point, so no bottleneck or extra hops.

3) Request Queue Management
⇒ Each process maintains a simple queue logic based on timestamps.
- Requests are handled on demand, reducing unnecessary messaging.

Page No.:
Date:

Q.10 Explain Chandy-Misra-Hass algorithm for distributed deadlock detection.

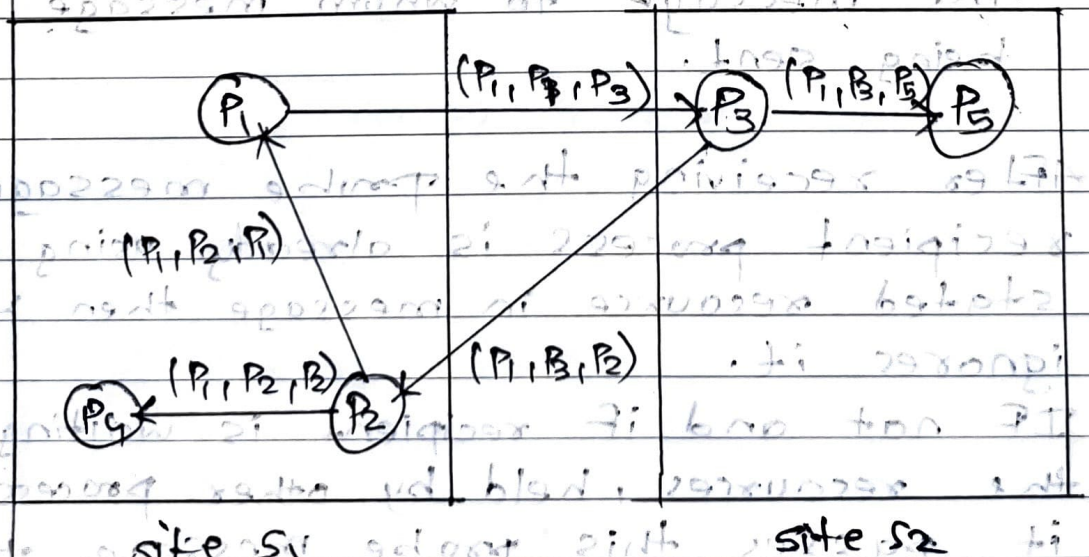
→ probe

- This is problem-based distributed system algorithm for detection of deadlock.
- This algorithm permits to processes to request multiple instances of resource type at a time.
- In this algorithm, when process fails to get requested single or multiple instances of resources within specified time, it sends probe message to processes holding that resource.
- Probe message contains following information:

1) Identifier of process just blocked.
2) Identifier of process sending this message.
3) Identifier of the recipient process of the message to whom message is being sent.

- After receiving the probe message if recipient process is already using the stated resource in message then it ignores it.
- If not and if recipient is waiting for the resources, held by other process then it passes this probe message to the processes holding this resources for which it is waiting.

- The recipient process of the probe message modifies the fields in the probe message and then forwards this message.
- These modifications are carried out in the following manner:
  1) First field is left unchanged.
  2) The recipient changes second field to its own process identifier.
  3) The third field is changed to the identifier of new recipient.
- In this way, every new recipient of the probe message repeats this changes in probe message.
- If the probe message returns back to the original sender of message then a cycle exists and system is deadlocked.

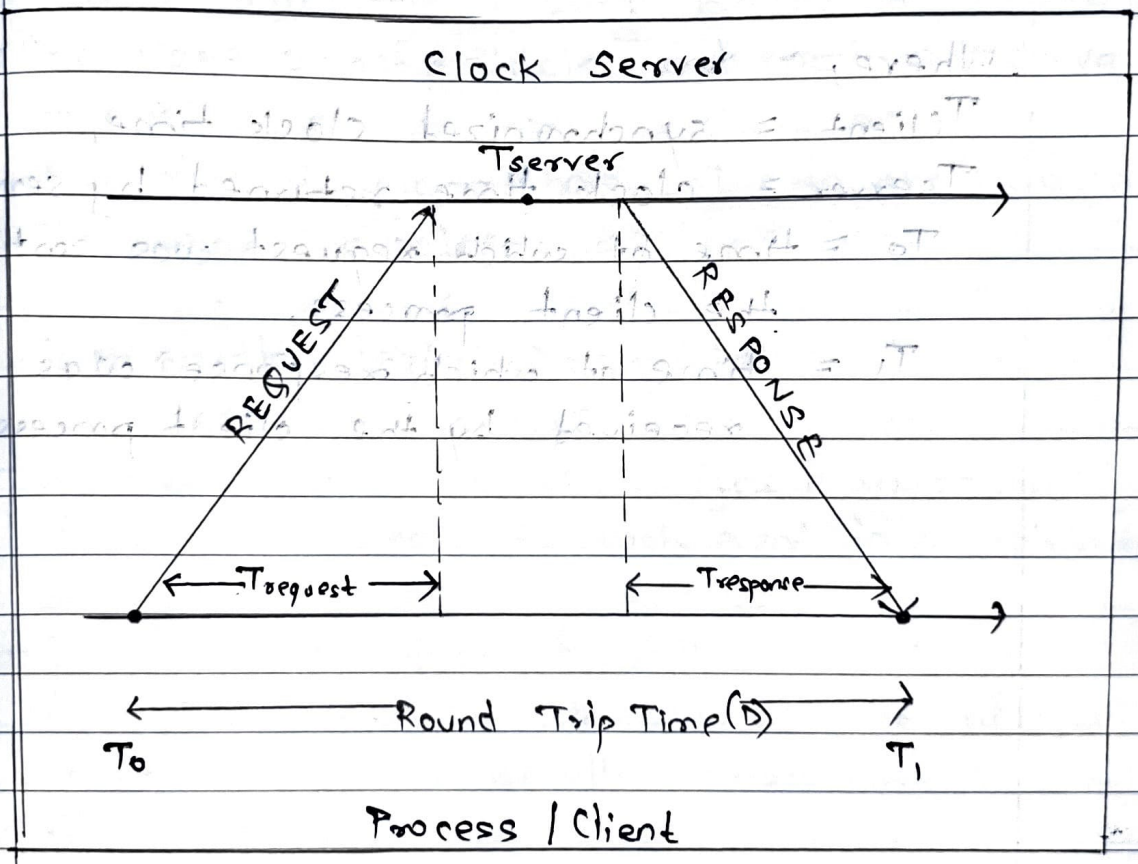


- consider the figure, in which process P_1 is blocked when it requests for the resource held by process P_3.
- Process P_1 ther sends probe message (P_1, P_1, P_3) to process P_3.
- After receiving this message process P_3 finds that it itself blocked on processes P_2 and P_5.
- As process P_5 is not blocked ~~by~~ on any process, it ignores the probe message (P_1, P_3, P_5) sent by P_3.
- Process P_2 ~~receives~~ ~~sends~~ receives probe message (P_1, P_3, P_2) from P_3 and finds that it itself blocked on P_1 and P_4.
- Process P_2 sends probe message (P_1, P_3, P_1) to process P_1 and (P_1, P_3, P_4) to process P_4.
- As probe message returns to original sender P_1, cycle exists and system is deadlocked.

What do you mean by clock synchronization? Explain the Cristian Algorithm with example.

Cristian Algorithm :

- Cristian's algorithm is a clock synchronization algorithm is used to synchronize time with time server by client processes.



1) The process on the client machine sends the request for fetching clock time to the clock server at time (T_0).
2) The server listens to the request made by client process and returns the response in form of clock server time.

3) The client process fetches the response from the clock server at time (T_1) and calculates the synchronized client clock time using the formula given below.

$$T_{client} = \frac{T_{server} + (T_1 - T_0)}{2}$$

$$T_{client} = T_{server} + \frac{(T_1 - T_0)}{2}$$

Where,

T_{client} = synchronized clock time,

T_{server} = clock time returned by server.

T_0 = time at which request was sent by the client process,

T_1 = time at which response was received by the client process.